Module 1-JavaScript

Exercise 1

Problem Statement:
JavaScript Basics & Setup
Scenario: Set up your community portal to use JavaScript.
Objective: Configure environment and test basic script functionality.
Task:
• Use <script src="main.js"></script> in HTML
• Log "Welcome to the Community Portal" using console.log()
• Use an alert to notify when the page is fully loaded

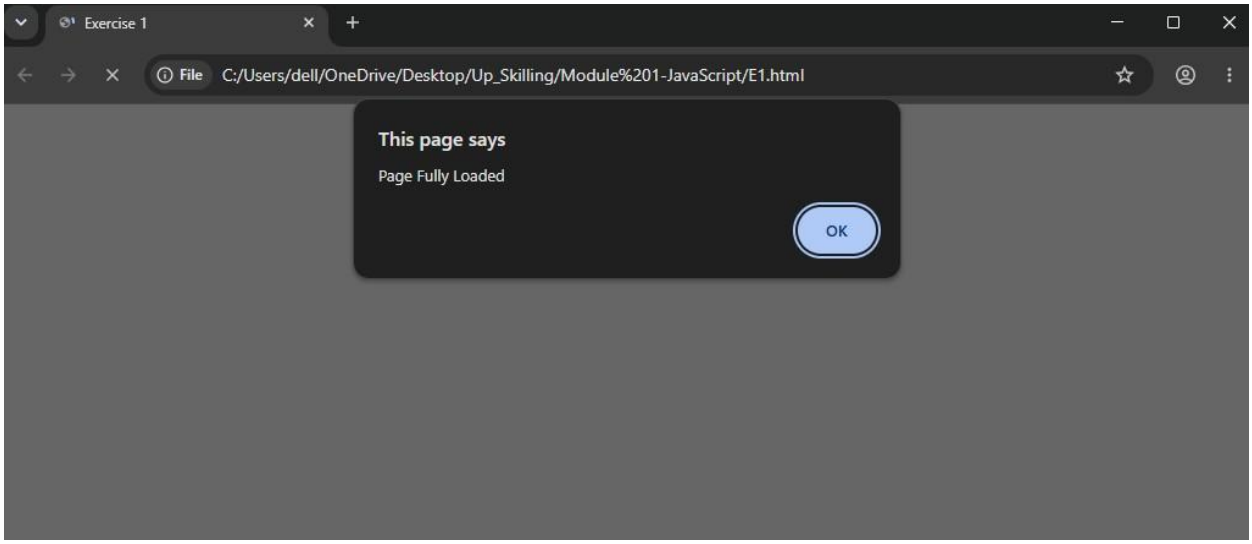
Code:
E1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Exercise 1</title>
</head>
<body>

<h1>Local Community Event Portal</h1>

<script src="main.js"></script>

</body>
</html>
```

E1.js

```
console.log("Welcome to the Community Portal");

window.onload = function () {
   alert("Page Fully Loaded");
};
```

Output:



Explanation:

- External JavaScript files are linked using the <script>tag.
- console.log()displays messages in the browser console, while alert()shows notifications.

Exercise 2

Problem Statement:
Syntax, Data Types, and Operators
Scenario: Store event details like name, date, and available seats.
Objective: Use proper data types and operations.
Task:
• Use const for event name and date, let for seats
• Concatenate event info using template literals
• Use ++ or -- to manage seat count on registration

Code:

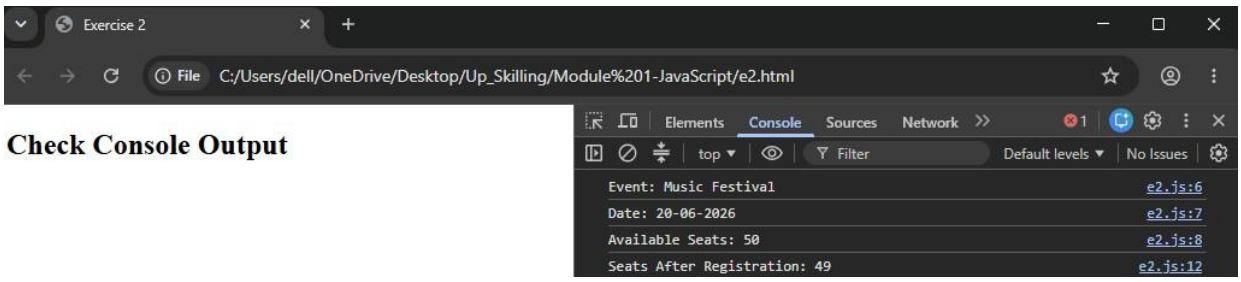
E2.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Exercise 2</title>
</head>
<body>

<h2>Check Console Output</h2>

<script src="main.js"></script>

</body>
</html>
```

E2.js

```
const eventName = "Music Festival";
const eventDate = "20-06-2026";

let seats = 50;

console.log(`Event: ${eventName}`);
console.log(`Date: ${eventDate}`);
console.log(`Available Seats:
${seats}`);

seats--;

console.log(`Seats After Registration: ${seats}`);
```

Output:



Explanation:

- constStores fixed values while letallows updates.
- Template literals simplify string concatenation and variable display.

Exercise 3

Problem Statement:
Conditionals, Loops, and Error Handling
Scenario: Only show valid events and limit registrations.
Objective: Apply conditions and handle invalid data.
User Story: As a user, I want only upcoming events with seats to be displayed.
Task:
• Use if-else to hide past or full events
• Loop through the event list and display using forEach()
• Wrap registration logic in try-catch to handle errors
Code:
E3.html

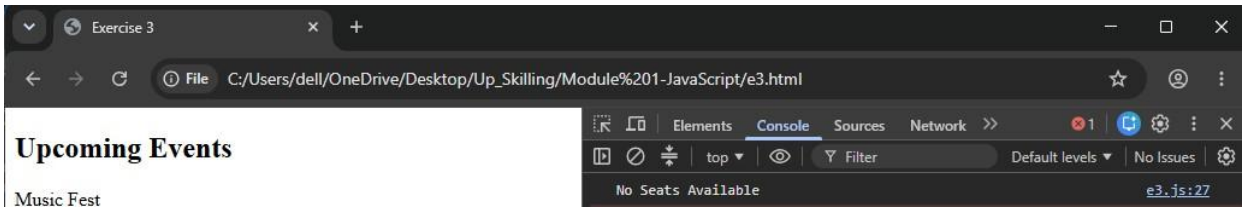
```
<!DOCTYPE html>
<html>
<head>
  <title>Exercise 3</title>
</head>
<body>

<h2>Upcoming Events</h2>

<div id="events"></div>

<script src="main.js"></script>

</body>
</html>
```

E3.js

```
const events = [
  {name:"Music Fest", seats:20, upcoming:true},
  {name:"Food Fair", seats:0, upcoming:true},
  {name:"Old Event", seats:15, upcoming:false}
];

events.forEach(event => {

  if(event.upcoming && event.seats > 0){
    document.getElementById("events").innerHTML +=
    `<p>${event.name}</p>`;
  }

});

try{

  let seats = 0;

  if(seats <=

  0){
    throw "No Seats Available";
  }
```

```
}
catch(error){

   console.log(error);

}
```

Output:



Explanation:

- Conditional statements filter valid events before display.
- try-catchhandles unexpected registration errors gracefully.

Exercise 4

Problem Statement:
Functions, Scope, Closures, Higher-Order Functions
Scenario: Create reusable functions for event operations.
Objective: Encapsulate logic and use closures.
Task:
• Create addEvent(), registerUser(), filterEventsByCategory()
• Use closure to track total registrations for a category
• Pass callbacks to filter functions for dynamic search
Code:
E4.html

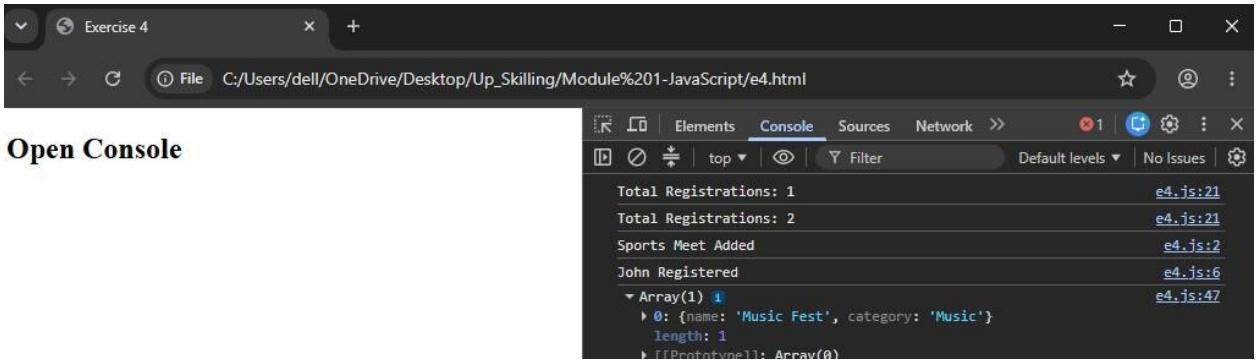
```
<!DOCTYPE html>
<html>
<head>
   <title>Exercise 4</title>
</head>
<body>

<h2>Open Console</h2>

<script src="main.js"></script>

</body>
</html>
```

E4.js

```
function addEvent(name){
   console.log(`${name} Added`);
}

function registerUser(user){
   console.log(`${user} Registered`);
}

function filterEventsByCategory(events, callback){
   return events.filter(callback);
}

function registrationTracker(){

   let count = 0;

   return function(){

      count++;

      console.log(`Total Registrations: ${count}`);

   };

}
```

```
const track = registrationTracker();

track();
track();

addEvent("Sports Meet");

registerUser("John");

const events = [
   {name:"Music Fest", category:"Music"},
   {name:"Sports Day", category:"Sports"}
];

const result =
filterEventsByCategory(
events,
event => event.category === "Music"
);

console.log(result);
```

Output:



Explanation:

- Functions help organize reusable logic.
- Closures preserve variables even after outer functions finish execution.

.

Exercise 5

Problem Statement:
Objects and Prototypes
Scenario: Each event is an object with properties and methods.
Objective: Model real-world entities using objects.
Task:
• Define Event constructor or class
• Add checkAvailability() to prototype
• List object keys and values using Object.entries()
Code:
E5.html

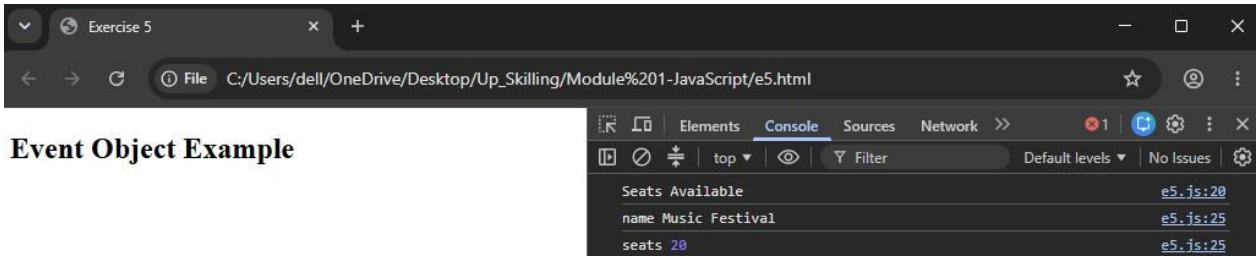
```
<!DOCTYPE html>
<html>
<head>
   <title>Exercise 5</title>
</head>
<body>

<h2>Event Object Example</h2>

<script src="main.js"></script>

</body>
</html>
```

E5.js

```
function Event(name,seats){

   this.name = name;
   this.seats = seats;

}

Event.prototype.checkAvailability =
function(){

   return this.seats > 0 ?
   "Seats Available" :
   "Full";

};

const event1 =
new Event("Music Festival",20);

console.log(event1.checkAvailability());

Object.entries(event1).forEach(
([key,value]) => {

   console.log(key,value);
```

});

Output:



Explanation:

- Objects model real-world entities using properties and methods.
- Prototypes allow methods to be shared among all object instances.

Exercise 6

Problem Statement:
Arrays and Methods
Scenario: Manage an array of all community events.
Objective: Use array methods for CRUD operations.
Task:
• Add new events using .push()
• Use .filter() to show only music events
• Use .map() to format display cards (e.g., "Workshop on Baking")
Code:
E6.html

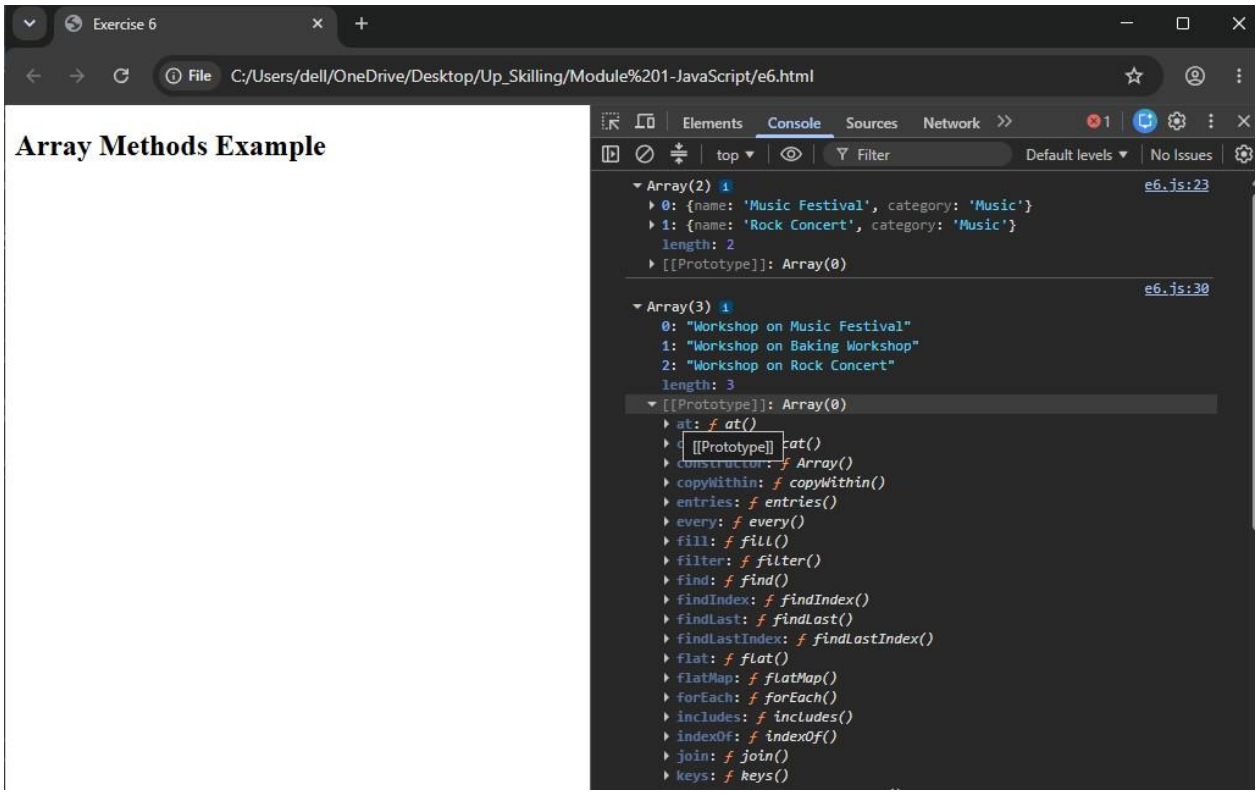
```
<!DOCTYPE html>
<html>
<head>
   <title>Exercise 6</title>
</head>
<body>

<h2>Array Methods Example</h2>

<script src="main.js"></script>

</body>
</html>
```

E6.js

```
let events = [];

events.push({
   name:"Music Festival",
   category:"Music"
});

events.push({
   name:"Baking Workshop",
   category:"Workshop"
});

events.push({
   name:"Rock Concert",
   category:"Music"
});

const musicEvents =
events.filter(
event => event.category === "Music"
);

console.log(musicEvents);

const displayCards
= events.map(
```

```
event => `Workshop on ${event.name}`
);

console.log(displayCards);
```

Output:



Explanation:

- Arrays store multiple event records efficiently.
- Methods like push(), filter(), and map()simplify data manipulation.

Exercise 7

Problem Statement:
DOM Manipulation
Scenario: Display all events dynamically on the webpage.
Objective: Render events using JS.
Task:
• Access DOM elements using querySelector()
• Create and append event cards using createElement()
• Update UI when user registers or cancels
Code:
E7.html

```
<!DOCTYPE html>
<html>
<head>

<title>Exercise 7</title>

<link rel="stylesheet" href="styles.css">

</head>
<body>

<h1>Community Events</h1>

<button onclick="registerEvent()">
Register
</button>

<button onclick="cancelEvent()">
Cancel Registration
</button>

<div id="eventContainer"></div>

<script src="main.js"></script>

</body>
</html>
```

E7.css

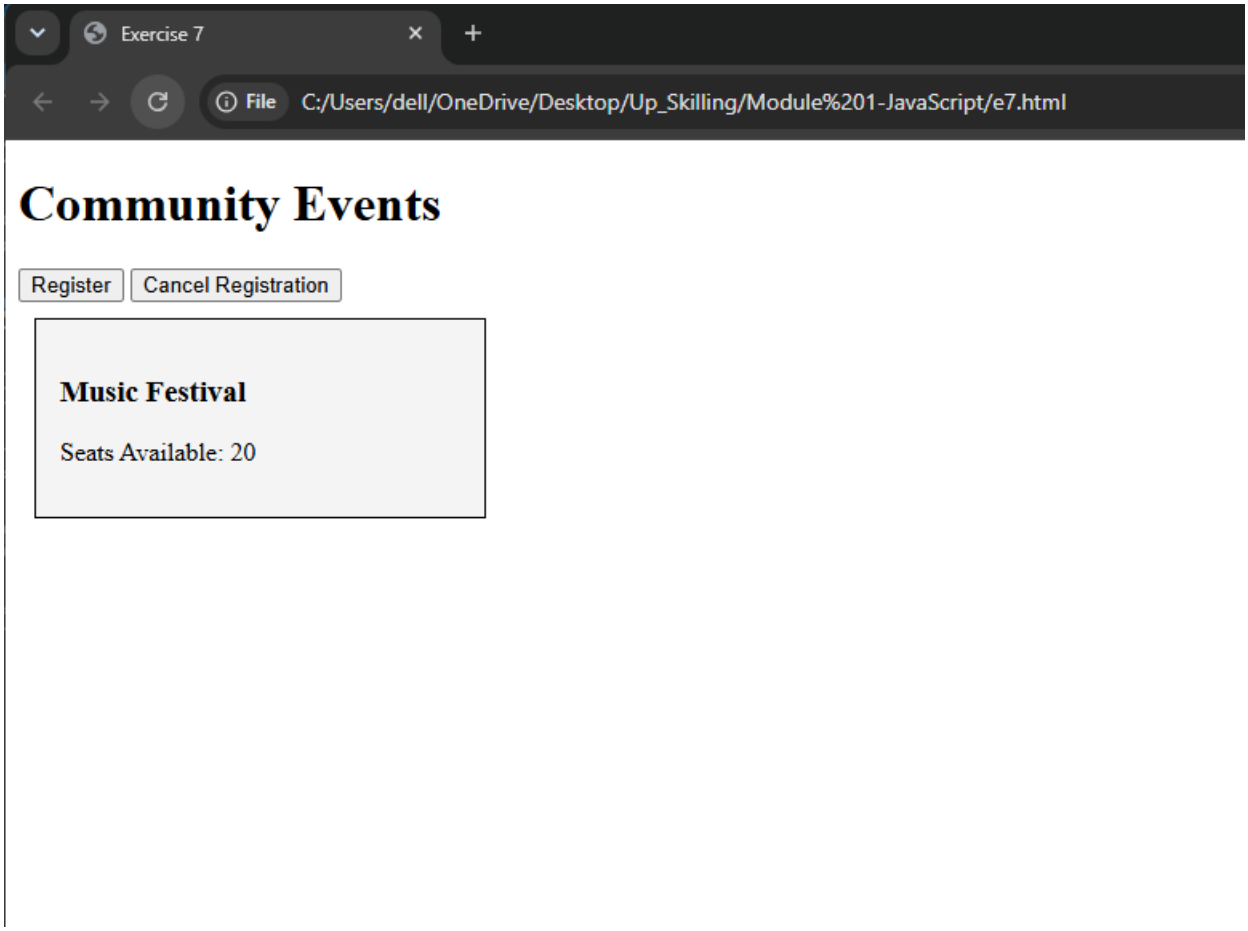
```
.eventCard{
   border:1px solid black;
   padding:15px;
   margin:10px;
   width:250px;
   background-color:#f4f4f4;
}
```

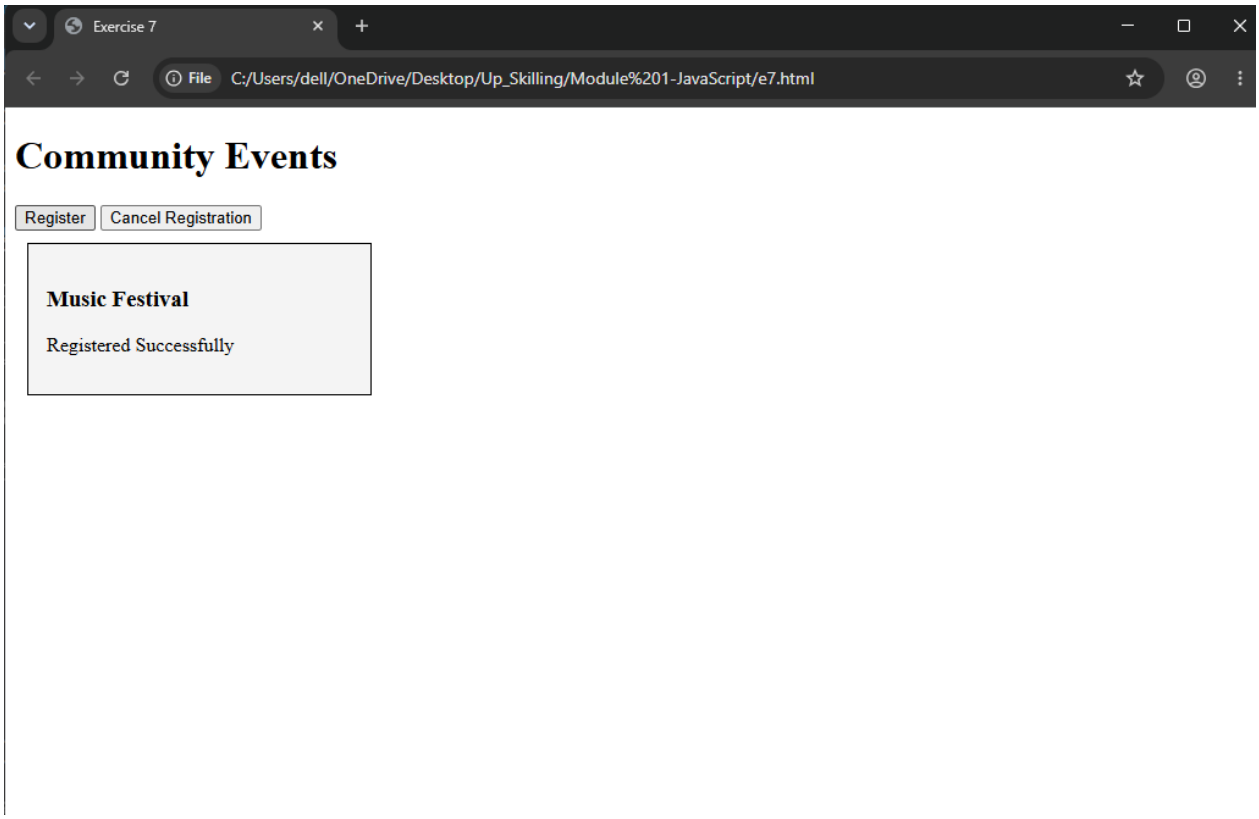
E7.js

```
const container =
document.querySelector("#eventContainer");

const card =
document.createElement("div");
```

```
card.className = "eventCard";

card.innerHTML =
"<h3>Music Festival</h3><p>Seats Available:

20</p>"; container.appendChild(card);

function registerEvent(){

   card.innerHTML =
   "<h3>Music Festival</h3><p>Registered Successfully</p>";

}

function cancelEvent(){

   card.innerHTML =
   "<h3>Music Festival</h3><p>Registration Cancelled</p>";

}
```

Output:


Exercise 7
File
C:/Users/dell/OneDrive/Desktop/Up_Skilling/Module%201-JavaScript/e7.html
Community Events
Register
Cancel Registration
Music Festival
Seats Available: 20



Explanation:

- DOM methods allow JavaScript to create and modify webpage content dynamically.
- User actions can instantly update event information without reloading the page.